

SMARTMOVE TRANSPORT SOLUTIONS

Data Management 2 - CW No 1

MEMBER 2 - FULL RESPONSIBILITIES, JOBS AND WORK PLAN

Primary role: PL/SQL and business-report specification, MongoDB theory/design, testing evidence, documentation, presentation and viva preparation.

Team model: Member 1 = system analysis and database design; Member 2 = technical theory/documentation and evidence; Member 3 = Oracle/PL/SQL/Spring Boot implementation; Member 4 = React/MongoDB implementation and UI.

This guide separates what Member 2 must personally produce from work that Member 2 must specify, review, or validate with the coding members.

Source basis. The assignment sheet requires an ER/EER diagram, a simple Web or Enterprise Application, an Oracle relational schema, nine core operations, realistic sample data, PL/SQL programs with exception handling, at least five business reports, MongoDB incorporation for specified dynamic information, four MongoDB query demonstrations, application-layer MongoDB integration, a presentation, and a software application with database implementation.

1. MEMBER 2 - ROLE DEFINITION

Member 2 is the bridge between the project design and the final evidence. The role is not simply “make PowerPoint slides”. Member 2 owns the specifications and explanations that tell the technical members what to implement and later proves, through documentation and demonstrations, that the implementation satisfies the CW.

The assessment explicitly requires PL/SQL procedures, functions, cursors and/or triggers with exception handling, plus at least five business reports. It also requires MongoDB to hold dynamic/unstructured transportation information and asks the team to demonstrate four query scenarios.

Member 2 therefore has six major work areas:

Area	Member 2 responsibility
A. PL/SQL planning	Define the required business logic, procedure/function/trigger/cursor scope, rules and exception cases for Member 3.
B. Business reports	Specify all five reports, their inputs, columns, calculations, expected outputs and business meaning.
C. MongoDB design theory	Specify collections, document examples, schema-flexible fields and the four required query demonstrations.
D. Testing and evidence	Create test cases, expected results, evidence checklist and final requirement-traceability checks.
E. Presentation	Own presentation structure, speaker notes, screenshots, demo sequence and final evidence organization.
F. Viva preparation	Build the team's question bank and ensure every member can explain the architecture, DBs, PL/SQL, reports and inte

Important: the coding members still implement the software. Member 2 defines the specification, checks correctness, records evidence, and owns the explanation/documentation side.

2. MASTER DELIVERABLES - WHAT MEMBER 2 MUST HAND OVER

By the end of the CW, Member 2 should have created or maintained the following project assets.

Deliverable	Minimum content	Used by
PL/SQL specification	Procedure/function/trigger/cursor list, purpose, inputs/outputs, rules, exceptions	Application M1
Report specification pack	Five report definitions, filters, columns, formulas, expected result	M3 + M4
MongoDB design pack	Collection purpose, document examples, flexible fields, references	M3 + M4
MongoDB query pack	Four required query scenarios and expected outputs	M4 + presentation
Test plan	Test IDs, input, expected result, actual result, status	M1 + M3 + M4
Requirement traceability	Every CW requirement mapped to implementation/evidence/owner	Whole team
Evidence pack	Screenshots/results for Oracle, PL/SQL, reports, MongoDB, UI and integration	Presentation/Viva
Presentation	Slides, speaker notes, screenshots and demo order	Whole team
Viva question bank	Conceptual and project-specific questions with understood answers	Whole team

Do not invent a separate mandatory printed report unless the lecturer separately instructs the group to submit one. The sheet lists Presentation and Software Application with Database Implementation as deliverables and says submission is a GitHub link.

3. WORK AREA A - PL/SQL BUSINESS LOGIC SPECIFICATION

The coding implementation belongs primarily to Member 3, but Member 2 should define exactly what the implementation is expected to do. This prevents weak or decorative PL/SQL.

3.1 Build the PL/SQL feature catalogue

Type	Recommended business operation	What Member 2 defines
Procedure	REGISTER_VEHICLE	Purpose, input fields, validation rules, success result, failure cases
Procedure	REGISTER_PASSENGER	Required information and duplicate-data behavior
Procedure	SCHEDULE_TRIP	Route/vehicle/driver validation and scheduling rules
Procedure	CREATE_BOOKING	Passenger/trip validation, seat availability and booking creation
Procedure	PROCESS_PAYMENT	Booking validation, payment result and error cases
Procedure	RECORD_MAINTENANCE	Vehicle validation and maintenance status logic
Procedure	ASSIGN_DRIVER	Driver/trip availability rules
Function	CALCULATE_REVENUE	Inputs, return value and date-range behavior
Function	GET_AVAILABLE_SEATS	Trip input and seat calculation
Function	CALCULATE_OCCUPANCY	Capacity/bookings calculation and division-by-zero handling
Trigger	Status/business rule update	When it fires and exactly what state should change
Cursor	Report/detail iteration	What rows are traversed and why a cursor is useful

3.2 Define business rules before coding

- A booking should not be created for a nonexistent passenger or trip.
- A booking should not exceed available seats.
- A payment should relate to a valid booking.
- A maintenance record should relate to a valid vehicle.
- Driver and vehicle assignment must respect availability rules decided by the team.
- Reports must use actual stored data rather than hard-coded values.

These are implementation recommendations for a coherent system. The assessment itself requires the business operations, PL/SQL programming and reports, but does not prescribe these exact procedure names or business rules. Keep the team's final rules consistent with its agreed design.

4. WORK AREA A2 - EXCEPTION-HANDLING DESIGN

The rubric's higher band calls for robust PL/SQL logic and advanced exception handling. Member 2 should therefore define meaningful failure cases rather than accepting a generic error handler everywhere.

■filecite■turn1file1■

Create an exception catalogue:

Case	Example situation	Expected response
Passenger not found	Booking references invalid passenger	Abort operation and return a clear message
Trip not found	Booking references invalid trip	Abort operation and return a clear message
No seats available	Requested booking exceeds remaining capacity	Reject booking without partial data
Vehicle unavailable	Trip is scheduled against an unavailable vehicle	Reject or require correction according to project rule
Booking not found	Payment/cancellation references unknown booking	Abort safely
Invalid payment	Payment violates agreed business validation	Do not mark booking as successfully paid
Duplicate data	Unique business key already exists	Return an understandable duplicate-data error
Unexpected error	Unexpected Oracle error	Log/return safe error information; do not silently ignore it

Member 2's job is to define the expected behavior and verify that Member 3 actually demonstrates the exception. A screenshot of only the successful path is not enough for strong evidence of exception handling.

Suggested evidence set:

- One screenshot showing successful execution of an important procedure.
- One screenshot or captured output showing a deliberate invalid-input case.
- One screenshot showing the final database state after the valid operation.
- A short explanation of why the exception exists and how partial updates are prevented.

5. WORK AREA B - FIVE BUSINESS REPORTS

The CW requires at least five business reports. Four examples are explicitly listed in the sheet; Member 2 should lock the fifth so implementation is unambiguous. ■filecite■turn1file0■

Recommended final set:

#	Report	Required content / specification
1	Most Frequently Used Routes	Route, booking count, passenger count and ranking/order.
2	Total Revenue Within a Given Period	Start date, end date, tickets sold, total revenue and optionally average ticket value.
3	Passenger Travel History	Passenger identity, trips/routes, travel dates, vehicle, seat/booking/payment details as available.
4	Vehicles Due for Maintenance	Vehicle, last/next maintenance information, current status and due logic.
5	Trip Occupancy & Performance	Trip, route, capacity, booked seats, available seats, occupancy percentage and revenue.

5.1 For every report, write a one-page specification with these headings:

- **Business purpose:** what decision the report supports.
- **Inputs/filters:** dates, passenger ID, trip, route or other filters.
- **Data sources:** Oracle tables/views/procedures/functions used.
- **Columns:** exact output fields and order.
- **Formula/calculation:** how totals, counts, rankings or percentages are calculated.
- **Expected result:** what a correct result should look like with the sample data.
- **Application presentation:** table/chart/filter idea for the React interface.

6. REPORT-BY-REPORT SPECIFICATION SHEETS

Report 1 - Most Frequently Used Routes

- Purpose: identify routes with the highest ticket-booking usage.
- Recommended inputs: optional date range or trip period, if the team wants time-based analysis.
- Core output: route, total bookings, ranking; optional passenger count.
- Expected ordering: highest usage first.
- Validation: use sample data containing multiple bookings across at least several routes.

Report 2 - Total Revenue Within a Given Period

- Purpose: calculate ticket-sales revenue over a selected period.
- Inputs: start date and end date.
- Core output: ticket count and total revenue; optional average ticket value.
- Validation: include transactions both inside and outside the selected range to prove filtering works.
- Edge case: reversed or invalid date range must be handled clearly.

Report 3 - Passenger Travel History

- Purpose: show a passenger's historical journeys.
- Input: passenger identifier selected by the user.
- Core output: passenger, trip, route, date, vehicle and booking/payment details available in the schema.
- Validation: test a passenger with multiple trips and one with no travel history.
- Expected behavior: no-history case should show a clear empty result, not an application error.

Report 4 - Vehicles Due for Maintenance

- Purpose: identify vehicles whose maintenance is due according to the agreed date/status rule.
- Possible logic: due date reached, maintenance interval reached, or explicit maintenance status.
- Core output: vehicle, maintenance history/last maintenance, due information and current status.
- Validation: include both due and not-due vehicles in sample data.
- Important: write down the exact "due" rule so the whole team explains the same logic in viva.

Report 5 - Trip Occupancy & Performance

- Purpose: compare trip capacity, bookings and revenue.
- Core output: trip, route, capacity, booked seats, available seats, occupancy percentage and revenue.
- Example formula: $\text{occupancy \%} = \text{booked seats} / \text{capacity} * 100$.
- Validation: test a trip with zero bookings, partial occupancy and full occupancy.
- Integration opportunity: combine structured occupancy/revenue from Oracle with customer-rating information from MongoDB in a hybrid dashboard, provided the team implements the linkage.

7. WORK AREA C - MONGODB THEORY AND DESIGN

The CW requires MongoDB as a complementary NoSQL database for unstructured and dynamic transportation-related information. It specifically names vehicle images/documents, passenger reviews and feedback comments, travel announcements/notifications, and trip-related multimedia content.

■filecite■turn1file0■

Member 2 must create the design that Member 4 can implement and Member 3 can expose through the application layer.

Collection	Purpose	Important design idea
vehicle_documents	Vehicle images and documents	Allow multiple files and flexible metadata without forcing all document types into one
reviews	Passenger reviews and feedback comments	Support rating, comment text, route/vehicle/driver references and flexible review
announcements	Travel announcements and notifications	Support message, publication time, audience/targeting and optional extra fields.
trip_media	Trip-related multimedia content	Support media metadata such as type, title/caption, URL/reference and trip linka

7.1 Schema-flexibility explanation

- Explain why dynamic fields are useful for reviews, notifications and multimedia metadata.
- Show at least two documents in the same collection with slightly different optional fields to demonstrate flexibility.
- Keep enough reference information to connect a MongoDB document back to the relevant route, vehicle, driver or trip in Oracle when the application needs a combined view.

Do not move the entire transactional system into MongoDB. The assignment describes MongoDB as a complementary NoSQL database while separately requiring an Oracle relational schema.

8. WORK AREA C2 - FOUR REQUIRED MONGODB QUERY DEMONSTRATIONS

Member 2 should write the scenario specification and expected output; Member 4 should implement the actual queries. The four scenarios below are explicitly required by the CW. ■filecite■turn1file0■

Query	What to demonstrate	Evidence to capture
1	Retrieve all passenger reviews for a specific route.	Query + result set showing route-specific reviews.
2	Identify highest-rated vehicles or drivers based on customer feedback.	Aggregation/sorting result with rating basis.
3	Search customer complaints or feedback using specific keywords.	Keyword query + matching comments/results.
4	Retrieve vehicle documents and multimedia information.	Document/media query + returned metadata/content references.

For each query, Member 2 should record:

- Business question.
- MongoDB collection used.
- Fields needed by the query.
- Query logic in plain English.
- Expected sample result.
- Screenshot/result evidence.
- How the result is surfaced through the application, where applicable.

9. WORK AREA D - TESTING AND QUALITY EVIDENCE

Member 2 owns the test-plan document and coordinates evidence. The coding members perform technical fixes; Member 2 records whether the requirement is actually demonstrated.

Recommended functional test catalogue:

ID	Area	Example test
T01	Authentication	Valid login succeeds; invalid login is rejected.
T02	Vehicle	Create/view/update vehicle through the application.
T03	Driver	Create/view/update driver information.
T04	Route	Create/view route and use it in trip scheduling.
T05	Passenger	Register/view passenger.
T06	Trip	Schedule a trip with valid route/vehicle/driver data.
T07	Booking	Book a trip when seats are available.
T08	Booking exception	Attempt to overbook a full/insufficient-capacity trip.
T09	Payment	Process a valid booking payment and verify state.
T10	Maintenance	Record maintenance and verify related status logic.
T11	Feedback	Submit feedback and verify storage/display.
T12	Report 1-5	Run each report with controlled data and verify output.
T13	Mongo Q1-Q4	Run all four required MongoDB query demonstrations.
T14	Integration	Open a UI screen that combines Oracle and MongoDB information.

Test-record format:

Test ID | Preconditions | Input | Expected Result | Actual Result | Status | Evidence File

10. WORK AREA D2 - REQUIREMENT TRACEABILITY MATRIX

This is one of Member 2's most valuable final checks. Every CW requirement should have a clear implementation owner and evidence. Use the following as the master checklist.

CW requirement	Owner / implementation	Member 2 evidence check
ER/EER diagram	M1	Confirm complete, normalized, readable version exists.
Web/Enterprise Application	M3 + M4	Confirm major workflows are functional and demonstrable.
Oracle relational schema	M1 design + M3 implementation	Confirm tables/relationships/sample data exist.
Vehicle Management	M3 + M4	Test and capture evidence.
Driver Management	M3 + M4	Test and capture evidence.
Route Management	M3 + M4	Test and capture evidence.
Passenger Management	M3 + M4	Test and capture evidence.
Trip Scheduling	M3 + M4	Test and capture evidence.
Ticket Booking	M3 + M4 + PL/SQL	Test happy and failure path.
Payment Processing	M3 + M4 + PL/SQL	Verify stored result/state.
Maintenance	M3 + M4 + PL/SQL	Verify due/status logic.
Feedback and Review	M3 + M4 + MongoDB	Verify storage and display.
Realistic sample data	M3 + M4	Verify reports have enough meaningful data.
PL/SQL programs + exceptions	M3, specified by M2	Collect successful and failure evidence.
Five business reports	M3 + M4, specified by M2	Verify all five run and are displayed.
MongoDB collections	M4	Verify four required categories exist.
MongoDB queries 1-4	M4, specified by M2	Collect each query/result.
Application-layer MongoDB integration	M3 + M4	Show MongoDB content in app.
Presentation	All	Finalize slides and demo order.
Software + database implementation	All	Final repository and run instructions ready.

11. WORK AREA D3 - EVIDENCE PACK

Create a clean evidence folder so the presentation and viva do not depend on improvisation.

Recommended repository structure:

```
docs/evidence/ ■■■ 01-erd/ ■■■ 02-oracle/ ■■■ 03-plsql/ ■■■ 04-reports/ ■■■ 05-mongodb/ ■■■  
06-frontend/ ■■■ 07-integration/ ■■■ 08-testing/ ■■■ 09-presentation/
```

Evidence to collect:

- ERD final image/PDF.
- Oracle schema/tables and representative SQL/PLSQL outputs.
- Procedure/function/trigger execution evidence.
- Exception-handling evidence.
- All five business report outputs.
- All four MongoDB query outputs.
- Application screens for the nine core operations.
- A screen that visibly combines Oracle-backed and MongoDB-backed information.
- Testing results and fixed/retested failures.

File-naming rule:

Use predictable names such as R01_FrequentRoutes.png, R02_Revenue.png, MONGO_Q1_RouteReviews.png, TEST_T08_Overbooking.png. This makes slide building and viva demonstration much faster.

12. WORK AREA E - PRESENTATION OWNERSHIP

The assessment is Software | Presentation | VIVA, and the rubric gives Presentation & Viva 20%. The highest band expects excellent understanding, confident communication and deep knowledge. ■filecite■turn1file0■ ■filecite■turn1file1■

Member 2 should own the presentation file and coordinate the final speaking flow, while all four members contribute and understand the content.

Slide area	Main presenter	Member 2 preparation job
Problem + objectives	M1	Make wording consistent with CW scenario.
Requirements	M1	Check every listed requirement appears.
ERD + normalization	M1	Place final readable diagram and key explanation.
Oracle implementation	M3	Select representative evidence only.
PL/SQL	M2 + M3	Show procedure/function/trigger/cursor concepts and one meaningful example per type used.
Five reports	M2 + M3	Show purpose and one result view for each report.
MongoDB	M2 + M4	Explain why complementary NoSQL storage is used.
Mongo queries	M4	Prepare all four demos/results.
Integration	M3 + M4	Show Oracle-backed + Mongo-backed information in one application flow.
Testing	M2	Show key tests and exception evidence.
Conclusion/innovation	Team	Tie system value to requirements without inventing unsupported claims.

13. WORK AREA E2 - LIVE DEMO SCRIPT

Prepare a deterministic demo path. The purpose is to prove the system works, not to explore the UI randomly.

Recommended sequence:

Step	Demo action	Evidence/value proved
1	Login to the system	Application readiness and role-based entry if implemented.
2	Open vehicle details	Oracle vehicle data plus MongoDB image/document/review content.
3	Create/view route and schedule trip	Route + trip scheduling.
4	Book a trip	Passenger + trip + booking workflow.
5	Process payment	Payment workflow and stored result.
6	Submit/view review	MongoDB feedback integration.
7	Open maintenance	Maintenance operation and due/status logic.
8	Run five reports	PL/SQL business-report requirement.
9	Demonstrate Mongo queries	All four required MongoDB scenarios.
10	Show hybrid page/dashboard	Oracle + MongoDB integration.

Have backup screenshots/video evidence for every major demonstration in case the live environment fails. This is a project-management recommendation, not an explicit submission requirement.

14. WORK AREA F - VIVA QUESTION BANK

Member 2 should prepare questions in two levels: general concepts and project-specific decisions.

14.1 Core concept questions

- Why is Oracle suitable for the transactional relational part of this system?
- Why does the project also use MongoDB?
- What is normalization and why was it applied?
- What is a stored procedure?
- What is a function and how is it different from a procedure?
- What is a cursor and where does the project use one?
- What is a trigger and why is it useful for a business rule?
- What is exception handling in PL/SQL?
- What is the difference between structured relational data and dynamic/unstructured content?
- Why should reports be based on real database data rather than hard-coded values?

14.2 Project-specific questions

- Explain the booking workflow from the React screen to the database.
- Which PL/SQL operation prevents overbooking?
- How is payment linked to a booking?
- How is "vehicle due for maintenance" determined?
- How does the application retrieve a review for a route?
- Which data is stored in Oracle and which data is stored in MongoDB, and why?
- How is a MongoDB document associated with an Oracle vehicle/trip/route?
- What happens when a required passenger or trip does not exist?
- How were the five reports validated?
- What does the application demonstrate that a database-only project would not?

For every answer, understand the actual project implementation. Do not memorize generic definitions that conflict with what your team built.

15. MEMBER 2 WORK ORDER - STEP BY STEP

Phase	Member 2 action	Output / exit condition
1. Requirement lock	Read the CW line by line and build a checklist.	No requirement is unassigned.
2. PL/SQL design	Define business operations, program types, rules and exceptions.	M3 has an exact implementation checklist.
3. Report design	Define all five reports and their inputs/output.	M3 can implement reports without guessing.
4. Mongo design	Define four collections and example documents.	M4 has a clear MongoDB target.
5. Query design	Define all four required query scenarios.	M4 has exact demo requirements.
6. Test planning	Create test IDs and expected results.	Testing can begin before the final demo.
7. Review build	Review implementation against the specifications.	Specification gaps are identified early.
8. Evidence capture	Collect screenshots/results and record test status.	Every major requirement has evidence.
9. Traceability	Map each CW requirement to code/UI/database/evidence.	Nothing is forgotten.
10. Presentation	Build final slides and speaking order.	Presentation tells one coherent story.
11. Viva	Create question bank and run practice sessions.	All members can explain the system.
12. Final QA	Run the complete checklist before GitHub submission.	Repository, demo and evidence are consistent.

16. FINAL MEMBER 2 CHECKLIST

Do not consider your work finished until every item below is checked.

Check	Done?
CW requirement checklist is complete.	■
PL/SQL specification is complete and handed to M3.	■
Procedure list is agreed.	■
Function list is agreed.	■
Trigger list is agreed.	■
Cursor usage is agreed.	■
Exception catalogue is agreed.	■
Five reports are fully specified.	■
Five report outputs have been tested.	■
Four MongoDB query scenarios are documented.	■
Four MongoDB query outputs are captured.	■
MongoDB collection design matches the actual implementation.	■
Test cases are executed and recorded.	■
Failed tests were fixed and retested.	■
Requirement-to-evidence traceability is complete.	■
Presentation screenshots match the final software.	■
Demo order is rehearsed.	■
Backup screenshots/evidence exist for critical demos.	■
Viva question bank is complete.	■
All four members can explain the full architecture and their own parts.	■
GitHub README and run instructions are consistent with the actual project.	■
No unsupported claims or invented requirements are presented as lecturer requirements.	■

17. WHAT MEMBER 2 SHOULD HAND TO EACH MEMBER

The best way to make the 2+2 structure work is to make every handoff explicit.

To Member 1:

- List of required business concepts that must remain consistent with the ERD and database design.
- Report/data requirements that depend on entity relationships.
- Any requirement ambiguity discovered during analysis.

To Member 3:

- PL/SQL operation catalogue.
- Exception catalogue and expected messages/behaviors.
- Five report specifications.
- Testing requirements for database/business logic.
- Evidence checklist for Oracle and PL/SQL.

To Member 4:

- MongoDB collection specifications.
- Example document structures.
- Four required MongoDB query scenarios.
- Frontend report presentation requirements.
- Testing and evidence requirements for MongoDB features.

To the whole team:

- Requirement traceability matrix.
- Final presentation structure.
- Demo script.
- Viva question bank.
- Final pre-submission checklist.

18. MEMBER 2 - FINAL RESPONSIBILITY STATEMENT

Your job is to make the project **provable**. Member 3 and Member 4 may build the software, but Member 2 makes sure the team can clearly answer: What did we build? Why did we build it this way? Which CW requirement does this feature satisfy? How is the business logic implemented? Where is the evidence? How does Oracle work with MongoDB? How were the reports validated?

The strongest outcome is not a large document. It is a project where the specification, implementation, screenshots, presentation and viva answers all tell the same story.

Target outcome: complete coverage of the CW requirements, strong business-report evidence, meaningful PL/SQL and MongoDB demonstrations, a coherent application story, and a team that can defend the implementation confidently during the viva.

Assessment grounding

The source assessment identifies the SmartMove scenario, required Oracle/PLSQL functionality, five business reports, MongoDB integration and four required MongoDB demonstrations, plus the presentation and software deliverables. Its marking rubric emphasizes complete implementation, robust business logic, MongoDB incorporation, and confident presentation/viva performance.

■filecite■turn1file0■L48-L91■ ■filecite■turn1file1■L92-L194■